

EXAMEN 2025 JUNIO

PARTE RECUPERACION INFORMACION (Muy similar al del 2024)

1. Tres términos: sistema AND operativo AND Windows. Explicar cómo se hace el proceso para hacer la consulta. Pregunta también el cómo se puede optimizar la query (Merge algorithm, Explicación, sin código).
2. Índice invertido y phrase query. Decir que son los números que aparecen y decir el resultado de la phrase query “to be” y “be to”

to:

1: 1,19,22; 2: 1,39,42;...

be:

1: 2,21,25; 2: 1,40,41;...

3. Un grafo y poner el porcentaje de ir a un nodo, y la probabilidad de transportarte es de 5%. Pregunta que obtengas el valor concreto para una fila de la matriz P en base al grafo presentado y las probabilidades indicadas.
4. Ejercicio práctico, daban dos queries y los resultados de estas. Calcular MRR y P@5 del sistema. Calcular el MAP del sistema (Daba el número de relevantes totales para cada query, que era mayor que el número de relevantes recuperados). Calcular el NCDG@3 para una de las queries (Daba valores de relevancia para cada documento 1, 3, 2, etc.). Se pueden expresar las ecuaciones con los valores numéricos concretos sin necesidad de resolverlas.
5. De una posting list decid que afecta eliminar una stopword del diccionario, eliminarla de la posting list y decir en nivel de tamaño que afecta más.
6. Explicar el porqué de aplicar normalización por longitud a los documentos.
7. Preguntas tipo test, 2 carillas de preguntas de respuesta única. 4 opciones una siempre de “ninguna de las anteriores”. Hay preguntas prácticas, que se tienen que resolver en base a un conjunto de pequeños documentos (3-5 palabras por documento). Estilo cual es el término de estos documentos con mayor IDF, que valor de TF/IDF será mayor para estos documentos y así. Algunas son puramente teóricas como una definición de HITS.

PARTE BASES DE DATOS NoSQL. 1 carilla y media

1. Esquema. Ventajas y desventajas del esquema. Cambios de esquema en base de datos relacionales (Nuevo y Legacy. Almacenar los cambios y los scripts de migración, mantener el soporte, etc.) y NoSQL (Esquema a nivel de aplicación, punto de acceso a datos, limitar el acceso a partes del agregado, la aplicación es la encargada de los cambios, datos que no se actualizan, caso de grafos, etc.).
2. Teorema CAP. Explicar el teorema, caso de sistema centralizado (Unica maquina) o caso de sistemas distribuidos. Compromiso entre latencia y consistencia a nivel real.
3. Explicar comandos/códigos de las prácticas. Los códigos no son los mismos pero siguen esta forma:

a.

```
mongod --replSet rsShard1 --port 27018
--dbpath /home/alumnogreibd/shard1 --bind_ip
localhost
```

b.

```
rs.initiate({
  _id: "rsConfServer",
  configsvr: true,
  members: [{_id: 0, host:
"localhost:27017"}]
})
```

c.

```
db.getMongo().setReadPref('primaryPref
erred')
```

d.

```
MERGE (m:Movie { title:"Cloud Atlas"
})
ON CREATE SET m.released = 2012
RETURN m;
```

EXAMEN 2024 JUNIO

PARTE RECUPERACION INFORMACION

8. Tres términos: sistema AND operativo AND Windows. Explicar cómo se hace el proceso para hacer la consulta.
9. Índice invertido y phrase query. Decir que son los números que aparecen y decir el resultado de la phrase query “to be” y “be to”

to:

1: 1,19,22; 2: 1,39,42;...

be:

1:2;20;70;5:30,50,23

10. Un grafo y poner el porcentaje de ir a un nodo, y la probabilidad de transportarte es de 5%.
11. Ejercicio práctico, daban dos queries y los resultados. Calcular MRR y P@5 del sistema. Calcular el MAP del sistema. Y luego algo del NCDG
12. De una posting list decid que afecta eliminar una stopword del diccionario, eliminarla de la posting list y decir en nivel de tamaño que afecta más.

PARTE BASES DE DATOS NO SQL. En una carilla las tres preguntas

4. Acceso a datos en NoSQL. Porque hay que tener en cuenta cómo se van a acceder a los datos cuando se hace el modelado.
5. Replicación: maestro-esclavo y peer-to-peer
6. Problema write-write, estrategias optimistas y pesimistas, almacenamiento de datos distribuidos y soluciones

Estos son los contenidos de RI que puso el profesor en el CV como “*Aspectos fundamentales*”. Lo que está de momento son apuntes que fui haciendo en clase, aún me faltan más por meter, el resto lo iré sacando de las diapositivas, cualquier aporte/modificación es bienvenida.

Conocer qué es Rec. Información (RI) y su diferencia con Rec. Estructurada (estilo bases de datos)

Recuperación de Información (RI): Encontrar material/información no estructurada que satisface una necesidad de información de entre una gran colección de datos.

- Necesidad de información: El concepto o idea que queremos obtener (ej.: cómo hacer un bizcocho)
- Query: Consulta concreta que se utiliza para filtrar la colección. (ej.: “bizcocho receta”)
- Colección / corpus: Conjunto de documentos
- Objetivo: Proporcionar documentos con la información relevante para la necesidad de información del usuario.
- Problema: La relevancia es subjetiva, dos usuarios pueden proporcionar la misma query, pero la información que se supone relevante para cada uno es diferente.
- Precisión: Fracción de elementos de la colección proporcionados/recuperados que son relevantes.
- Recall: Fracción de elementos relevantes de la colección que se proporcionan/recuperan a partir de la query.

Recuperación estructurada (RE): Recuperación de datos almacenados en BBDD estructuradas, donde la información está organizada en tablas con relaciones predefinidas entre los datos. La RE implica consultas a BBDD utilizando lenguajes como SQL para recuperar información específica de acuerdo con un esquema predefinido.

Volumen de datos: Existe una mayor cantidad de información no estructurada que de información estructurada ya que la primera es “gratis” de crear, los usuarios la crean de forma continua, mientras que las bases de datos requieren de un gran esfuerzo para ser diseñadas e implementadas.

Cuota de mercado: A mediados de los 90 la mayoría del mercado estaba orientado a las BBDD (estructurado), mientras que a día de hoy existen un gran número de tecnologías y corporaciones orientadas a la información no estructurada (Google, Microsoft / Bing).

Modelos booleanos de búsqueda

Las queries booleanas utilizan los operadores AND, OR y NOT. No existe incertidumbre, o se encuentra la palabra en el documento o no, pero no se realizan estimaciones de ningún tipo. Principal herramienta de recuperación de información por 3 décadas. Sigue en uso en multitud de sistemas. Se implementa especialmente en buscadores profesionales ya que el usuario sabe exactamente qué quiere.

Matriz de incidencia de término/documento: Una fila por término, una columna por documento, se almacena True si el término se encuentra en el documento, False en caso contrario.

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Es un método ineficiente para un elevado número de documentos, 1 millón de documentos con 500K términos distintos ocupa un trillón de 0s y 1s. La matriz es dispersa (está repleta de 0s) por lo que se malgasta mucho almacenamiento en representar las no apariciones.

Vectores de incidencia: Representaciones unidimensionales de documentos en función de los términos que contienen. Son útiles para calcular la similitud entre documentos y consultas, además de para operaciones de recuperación de información.

Incidence vectors

- So we have a 0/1 vector for each term.
- To answer query: take the vectors for **Brutus**, **Caesar** and **Calpurnia** (complemented) → bitwise **AND**.
 - 110100 **AND**
 - 110111 **AND**
 - 101111 =
 - **100100**

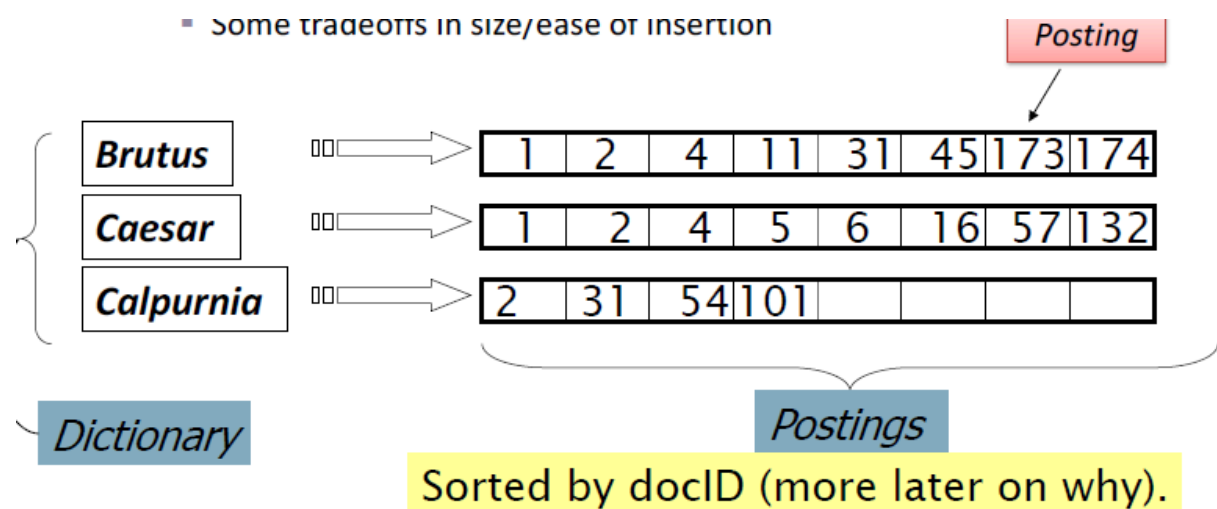
	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Índices invertidos. Entender el concepto y saber dibujar un índice invertido (con información posicional o sin ella) a partir de un corpus de ejemplo de texto.

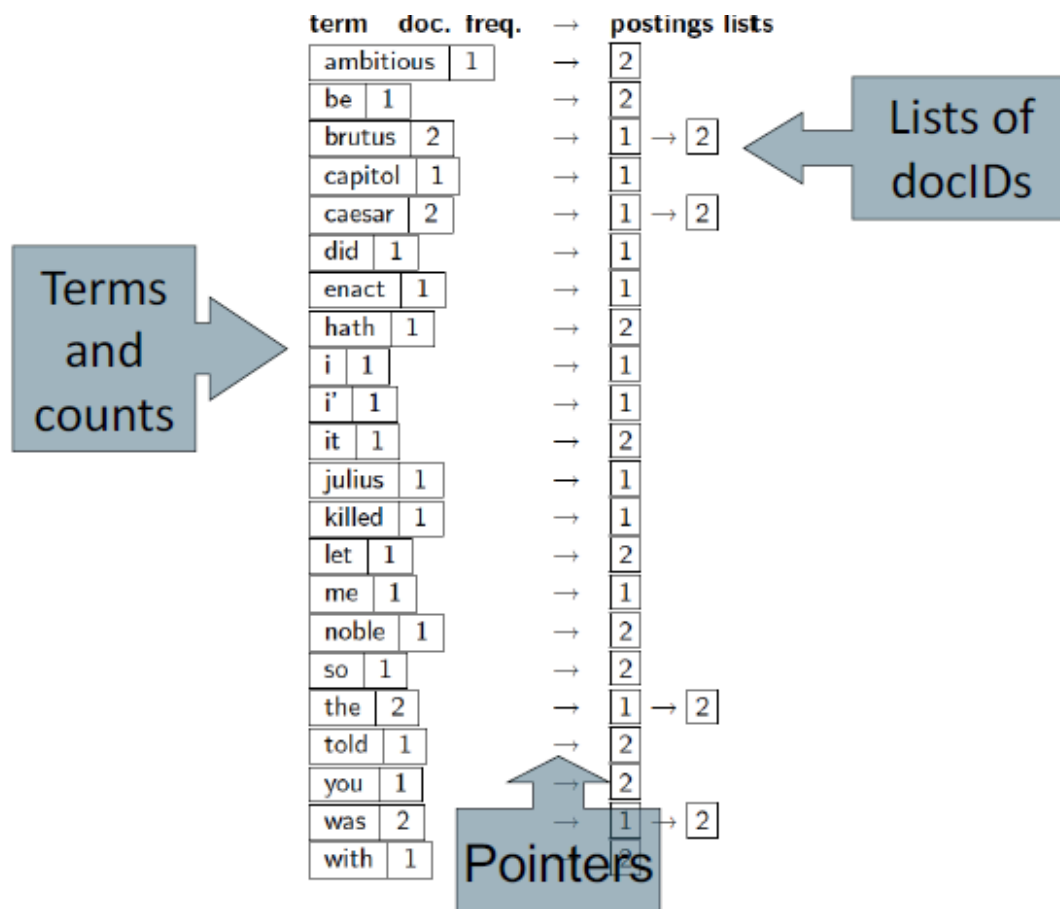
Estructura de datos que mapea términos de vocabulario a los documentos en los que aparecen. Esta estructura facilita la recuperación eficiente de documentos basada en términos de consulta.

Para cada término t se almacena una lista de documentos (posting list), representados por un docId, que contienen t . Es habitual que, además, se almacene el doc frequency (número de documentos en los que aparece).

Índice sin doc frequency



Índice con doc frequency



La posting list debe ser un array de tamaño variable para permitir la inserción de nuevos documentos en el corpus, además de que hay palabras mucho más frecuentes que otras por lo que se volvería a malgastar espacio. Debe estar ordenada por docId para mejorar la compresión y la búsqueda.

Preprocesado de texto (tokenización, normalización, stemming, stopwords removal)

Tokenización: Separar la cadena de caracteres en tokens.

Normalización: Modificación de los tokens para que tengan el mismo formato que los elementos de la query.

Stemming: Agrupación de palabras según semántica.

Stopword removal: Palabras muy comunes que se pueden omitir ya que aportan poco significado y consumen mucho almacenamiento (posting list). En web retrieval no se utiliza ya que puede llevar a resultados incorrectos.

Conceptos básicos de procesamiento de queries booleanas (intersección de posting lists).

Querys AND

Localizar los términos en el diccionario y obtener sus posting lists.

Intersecar las posting list (sólo permanecen los elementos que aparecen en ambas (todas) las posting list).

Como las posting list están ordenadas, el proceso de intersecar se vuelve más eficiente. En el peor de los casos la complejidad es $O(x + y)$ siendo x e y el tamaño de ambas posting list.

Uso de índice posicional para responder a phrase queries

Phrase Queries

Una phrase query es una consulta donde los términos deben aparecer juntos en el mismo orden en un documento, formando una frase exacta. Por ejemplo, la consulta "gato negro" solo debería devolver documentos donde "gato" y "negro" aparecen consecutivamente.

Cómo Responde un Índice Posicional a Phrase Queries

Para manejar phrase queries, el índice posicional permite verificar rápidamente si los términos aparecen en el orden correcto y en posiciones consecutivas en los documentos. El proceso general es:

a) *Búsqueda de Términos:*

- *Se buscan las listas de posiciones de cada término en la frase.*

Ejemplo: Para la frase "gato negro":

"gato" -> [(doc1, [3, 15, 28]), (doc3, [5, 19]), (doc4, [7])]

"negro" -> [(doc1, [4, 16, 29]), (doc3, [6, 20]), (doc4, [8])]

b) *Intersección de Posiciones:*

- Se intersectan las listas de posiciones para encontrar documentos donde los términos aparecen en las posiciones esperadas, es decir, "gato" en la posición p y "negro" en la posición $p+1$.

Ejemplo:

doc1: (3, 4), (15, 16), (28, 29)

doc3: (5, 6), (19, 20)

doc4: (7, 8)

Ventajas del Índice Posicional

- *Eficiencia:* Permite responder a consultas de frases de manera rápida y precisa.
- *Flexibilidad:* Facilita la búsqueda de frases exactas y términos adyacentes en un texto.

Indexado distribuido. Aproximación map reduce

Conceptos clave:

posting : un posting es un diccionario clave valor identificado por la palabra a buscar y un array de los doc Ids de los documentos que la contienen.

ejemplo:

“perro”, <doc1, doc2, doc3>

un posting list es un listado de postings.

Introducción a MapReduce

MapReduce es un modelo de programación utilizado para procesar y generar grandes conjuntos de datos de manera paralela y distribuida. Fue desarrollado por Google y presentado por Dean y Ghemawat en 2004. Su objetivo principal es facilitar el procesamiento de datos masivos utilizando un clúster de computadoras. El modelo de MapReduce se compone de dos fases principales: Map y Reduce.

Construcción de Índices Invertidos con MapReduce

La construcción de índices invertidos es un proceso crítico en los motores de búsqueda, ya que permite la recuperación rápida y eficiente de documentos. Para manejar grandes volúmenes de datos, como es común en la web, se utiliza el enfoque de MapReduce. A continuación, se describe cómo se aplica MapReduce en este contexto:

1. Etapa Map:

- Parsers: Cada documento del conjunto de datos se divide en fragmentos o splits. Cada fragmento es procesado por un nodo del clúster. Los nodos, conocidos como parsers, leen los documentos y extraen las palabras clave (tokens). Luego, emiten pares clave-valor en la forma de (término, docID).

Ejemplo:

Documento 1: "El gato se sienta en la alfombra"

Parsers Output: [("el", doc1), ("gato", doc1), ("se", doc1), ("sienta", doc1), ("en", doc1), ("la", doc1), ("alfombra", doc1)]

-
- 2. Etapa Shuffle and Sort:
 - Los pares clave-valor generados en la etapa Map se envían a diferentes nodos basándose en la clave (término). Este proceso se llama shuffle (reorganización). Luego, los nodos agrupan los pares por clave, de modo que todas las instancias de una palabra específica se agrupan juntas.
- 3. Etapa Reduce:
 - Inverters: En esta etapa, los nodos reciben todas las instancias de una palabra específica y las combinan para formar la lista de postings. Para cada clave (término), se crea una lista de documentos en los que aparece.

Ejemplo:

alfombra: [("doc1", 1), ("doc2", 1), ("doc3", 1)]

- Esto indica que la palabra "alfombra" aparece en los documentos 1, 2 y 3.
- 4. Mantenimiento del Diccionario y Postings List:
 - Un problema en la construcción del índice es mantener el diccionario (mapa de términos a termID). El enfoque SPIMI (Single-pass in-memory indexing) resuelve esto generando diccionarios separados para cada bloque y acumulando postings en listas conforme aparecen, sin necesidad de mantener un mapa global durante el procesamiento.

Ventajas del Enfoque MapReduce:

- Escalabilidad:
 - Permite procesar grandes cantidades de datos en paralelo, distribuyendo la carga entre múltiples nodos del clúster.
- Tolerancia a Fallos:
 - MapReduce maneja fallos de nodos re-ejecutando las tareas fallidas, asegurando la confiabilidad del procesamiento.
- Eficiencia:
 - La separación de tareas en dos fases (Map y Reduce) optimiza el uso de recursos, permitiendo manejar grandes volúmenes de datos de manera eficiente.

Ejemplo de Flujo de Datos en MapReduce:

Entrada:

Documento 1: "El gato se sienta en la alfombra"

Documento 2: "El perro salta sobre la alfombra"

Documento 3: "La alfombra es azul"

Map:

("el", doc1), ("gato", doc1), ("se", doc1), ("sienta", doc1), ("en", doc1), ("la", doc1), ("alfombra", doc1)

("el", doc2), ("perro", doc2), ("salta", doc2), ("sobre", doc2), ("la", doc2), ("alfombra", doc2)

("la", doc3), ("alfombra", doc3), ("es", doc3), ("azul", doc3)

Shuffle and Sort:

("alfombra", [doc1, doc2, doc3]), ("azul", [doc3]), ("el", [doc1, doc2]), ("gato", [doc1]), ("en", [doc1]), ("es", [doc3]), ("perro", [doc2]), ("salta", [doc2]), ("se", [doc1]), ("sienta", [doc1]), ("sobre", [doc2])

Reduce:

alfombra: [doc1, doc2, doc3]

azul: [doc3]

el: [doc1, doc2]

gato: [doc1]

en: [doc1]

es: [doc3]

perro: [doc2]

salta: [doc2]

se: [doc1]

sienta: [doc1]

sobre: [doc2]

Este enfoque asegura que el índice invertido se construya de manera eficiente, escalable y tolerante a fallos, permitiendo una recuperación de información rápida y precisa en grandes conjuntos de datos.

Ranked retrieval. Modelos de ponderación (tf/idf)

Ranked Retrieval

En sistemas de recuperación de información, "ranked retrieval" se refiere a la capacidad de clasificar documentos en función de su relevancia respecto a una consulta específica. A diferencia de las consultas Booleanas, donde un documento es relevante o no relevante, en el ranked retrieval se asigna una puntuación a cada documento. Esta puntuación determina el orden en que los documentos se presentan al usuario.

Modelos de Ponderación (tf-idf)

Para calcular estas puntuaciones y ordenar los documentos, se utilizan modelos de ponderación como el tf-idf. Este modelo combina dos factores principales:

1. **Term Frequency (tf):** Frecuencia de término
 - Definición: Número de veces que un término t aparece en un documento d .
 - Motivación: Cuantos más veces aparezca un término en un documento, más relevante es ese documento respecto al término.
 - La relevancia no crece proporcionalmente con el tf , por lo que habitualmente se pondera logarítmicamente.

$$w_{t,d} = \begin{cases} 1 + \log_{10} tf_{t,d}, & \text{if } tf_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

2. **Inverse Document Frequency (idf):** Frecuencia inversa de documentos
 - Definición: Se calcula como

$$idf(t) = \log \left(\frac{N}{df(t)} \right)$$

- donde N es el número total de documentos en la colección y $df(t)$ es el número de documentos que contienen el término t .
- Motivación: Reduce el peso de los términos muy comunes que aparecen en muchos documentos, ya que estos términos no ayudan a discriminar entre documentos relevantes y no relevantes.

Ponderación tf-idf

La ponderación tf-idf se calcula como:

$tf - idf(t, d) = tf(t, d) \times idf(t)$ o $w(t,d) \times idf(t)$ en caso de ponderar el tf.

- Ventajas:
 - Discriminación: Los términos que aparecen con frecuencia en un documento pero en pocos documentos del conjunto reciben más peso, ayudando a discriminar mejor la relevancia.
 - Relevancia: Los términos que son importantes para el documento pero no comunes en el corpus se destacan.

Ejemplo de Cálculo tf-idf

1. Term Frequency (tf):
 - Documento 1: "car": 3, "insurance": 2
 - Documento 2: "car": 1, "insurance": 5
2. Inverse Document Frequency (idf) (supongamos N=100 y df: "car" = 10 y "insurance" = 2):
 - "car": $idf = \log(\frac{100}{10}) = 1$
 - "insurance": $idf = \log(\frac{100}{2}) = 1.7$
3. **tf-idf**:
 - Documento 1: "car": $3 \times 1 = 3$, "insurance": $2 \times 1.7 = 3.4$
 - Documento 2: "car": $1 \times 1 = 1$, "insurance": $5 \times 1.7 = 8.5$

Puntuación de un documento para una query

$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$

- Modelo vectorial de Recuperación de Información

Modelo Vectorial

Los documentos y consultas se representan como vectores en un espacio multidimensional, donde cada dimensión corresponde a un término en el vocabulario.

1. Representación de Documentos:
 - Cada documento d se representa como un vector $d = (w_{t_1, d}, w_{t_2, d}, \dots, w_{t_n, d})$ donde $w_{t_i, d}$ es el peso tf-idf del término t_i en el documento d .
2. Representación de Consultas:

- Una consulta se puede representar de manera similar a un documento, como un vector de términos.

Similitud del Coseno

Para medir la similitud entre un documento y una consulta, se utiliza la similitud del coseno:

$$\text{sim}(d, q) = \cos(\theta) = \frac{(d \cdot q)}{(\|d\| \|q\|)}$$

$$\text{sim}(d_1, d_2) = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|},$$

- Producto punto: $d \cdot q$ Es la suma de los productos de los pesos de los términos comunes en el documento y la consulta.
- Normalización: $\|d\|$ y $\|q\|$ Son las magnitudes de los vectores del documento y la consulta, respectivamente, lo que normaliza la similitud para evitar que documentos más largos tengan una ventaja injusta.

- Normalización por longitud de documento

Un vector (documento) puede ser normalizado dividiendo cada uno de sus componentes por su longitud (normalización L_2). Esto hace que los vectores sean unitarios, por lo que dos documentos d y d' (siendo d' el documento d adjuntado a si mismo $d' = ddd$) tienen vectores idénticos.

- Modelo probabilística de RI (BIM, aspectos básicos, sin memorizar fórmulas). Saber estimar probabilidades p_i y r_i a partir de una tabla de ocurrencia de términos en docs relevantes e irrelevantes.

- BM25. Entender qué mejoras supone respecto a BIM.

El objetivo de BM25 es ser sensible a la TF sin añadir demasiados parámetros. Está basada en la distribución de Poisson. Se utilizan dos modelos de Poisson, uno para términos élite y otro para los que no lo son. Un término élite en un documento es aquel que está directamente relacionado con la temática del documento.

- Métricas de evaluación (y saberlas computar a partir de un conjunto o ranking de resultados): Precision, Recall, $P@k$, MAP, MRR, NDCG

- Evaluación mediante clicks (conceptos básicos).

- Interleaved rankings y A/B testing

- Clasificación de textos. Conceptos básicos. De qué tarea se trata la clasificación documental.

- *Métodos de clasificación (reglas, aprendizaje supervisado con Naive Bayes, knn, Rocchio, etc)*

- *Métricas de rendimiento de clasificación (accuracy, etc).*

- *Link analysis. Anchor text. PageRank. HITS*

- *Conceptos básicos de crawling*

Aspectos fundamentales de cara al examen de teoría de la parte de RI

- Conocer qué es Rec. Información (RI) y su diferencia con Rec. Estructurada (estilo bases de datos)

Recuperación de información es el proceso de encontrar material de naturaleza no estructurada en grandes colecciones para satisfacer una necesidad de información.

Recuperación Estructurada se refiere a la búsqueda de datos dentro de bases de datos relacionales, donde la información está organizada en tablas con filas y columnas

La diferencia es: la estructura de datos donde se realiza la búsqueda, el método de consulta: RI consultas booleanas y RE SQL, la relevancia de los resultados: RI puede variar en relevancia y utiliza rankings mientras que RE los resultados son exactos

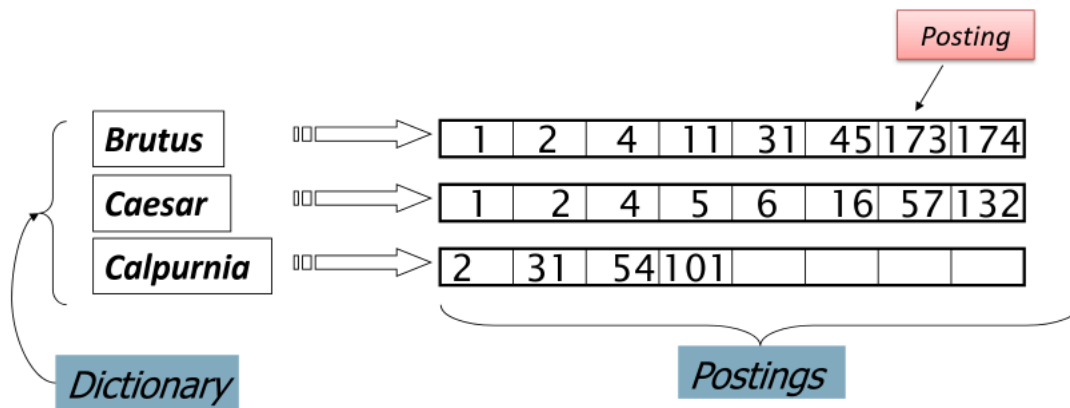
- Modelos booleanos de búsqueda

Son los 2 primeros:

- **Matriz de incidencia término por documento:** Es una matriz binaria con todas las palabras diferentes como filas y los documentos de la colección como columnas. Cada posición i, j de la matriz tiene valor 1 si el documento de la columna j contiene la palabra de la fila i o 0 si no lo contiene.

	<i>Antony and Cleopatra</i>	<i>The Tempest</i>	<i>Hamlet</i>	<i>Macbeth</i>
Antony	1	0	0	1
Brutus	1	0	1	0
Caesar	1	0	1	1
Calpurnia	0	0	0	0
Cleopatra	1	0	0	0
mercy	1	1	1	1
worser	1	1	1	0

- **Estructura indexada - índices invertidos:** consiste en indicar únicamente los documentos en los que sí está presente cada palabra

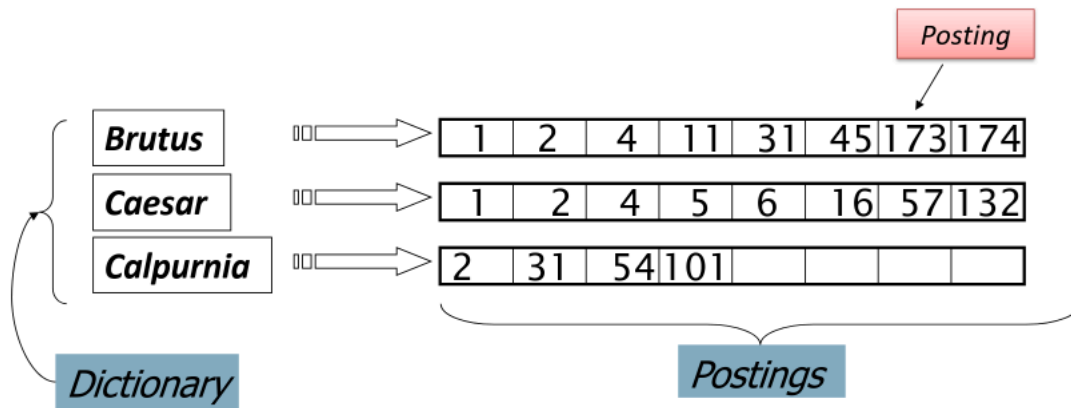


- **Procesamiento de consultas booleanas:** La operación de intersección es una de las más importantes para esta tarea. Es muy importante realizar la intersección de las postings lists para poder encontrar rápidamente los documentos que contengan los términos indicados.

- **Phrase queries:** Las phrase queries consisten en un conjunto de palabras que deben estar juntas en el documento en el que se buscan.

- **Índices invertidos.** Entender el concepto y saber dibujar un índice invertido (con información posicional o sin ella) a partir de un corpus de ejemplo de texto.

Esta estructura se basa en un conjunto de términos que se puede denominar diccionario, vocabulario o lexicon. Este conjunto contiene las diferentes palabras existentes en la colección ordenadas alfabéticamente (para facilitar la búsqueda). Cada palabra tiene asociada una lista de todos los documentos en los que aparece, llamada **lista de ocurrencias** o **postings list**.



Índice Invertido sin Información Posicional

Este tipo de índice simplemente mapea cada término a los documentos en los que aparece, sin almacenar la posición exacta dentro del documento.

Ejemplo

Consideremos un corpus de dos documentos:

- Documento 1: "la casa verde"
- Documento 2: "la casa azul"

El índice invertido sin información posicional sería:

Término	Documentos
la	1, 2
casa	1, 2
verde	1
azul	2

Índice Invertido con Información Posicional

Este tipo de índice no solo mapea cada término a los documentos en los que aparece, sino también a las posiciones específicas dentro de esos documentos.

Ejemplo

Usando el mismo corpus:

- Documento 1: "la casa verde"
- Documento 2: "la casa azul"

El índice invertido con información posicional sería:

Término	Documentos y posiciones
la	1:(1), 2:(1)
casa	1:(2), 2:(2)
verde	1:(3)
azul	2:(3)

Aquí, los números entre paréntesis indican la posición del término en el documento. Por ejemplo, "la" aparece en la posición 1 en ambos documentos.

- Preprocesado de texto (tokenización, normalización, stemming, stopword removal)

- Tokenización: La tokenización es el proceso de dividir un texto en unidades más pequeñas, llamadas "tokens", que pueden ser palabras, frases, o incluso caracteres.

- Normalización: La normalización incluye varias técnicas para convertir los tokens a una forma estándar. Esto puede involucrar convertir todos los caracteres a minúsculas, eliminar puntuación, o expandir contracciones.

- Stemming: una técnica para reducir las palabras a sus raíces o formas base. Reduce las palabras a su raíz morfológica. No siempre produce palabras reales.

- Stopword removal: Las stopwords son palabras comunes que generalmente no aportan mucho significado y se eliminan del texto para reducir el ruido. Ejemplos de stopwords en español incluyen "la", "y", "de", etc.

- Conceptos básicos de procesamiento de queries booleanas (intersección de posting lists).

El procesamiento de consultas booleanas es un enfoque fundamental en la recuperación de información que permite realizar búsquedas basadas en

operadores lógicos como AND, OR y NOT. Uno de los conceptos clave en este contexto es la intersección de posting lists.

La intersección se refiere al uso de algoritmos de combinación (merge algorithms), que son un conjunto de algoritmos que combinan listas ordenadas, en este caso se utilizarían con el operador AND. Es muy importante realizar la intersección de las postings lists para poder encontrar rápidamente los documentos que contengan los términos indicados.

Posting List: una posting list es una lista de documentos en los que aparece un término específico. Cada entrada en una posting list indica que un documento contiene el término correspondiente. La operación de intersección es una de las más importantes para esta tarea. Es muy importante realizar la intersección de las postings lists para poder encontrar rápidamente los documentos que contengan los términos indicados. También se conoce esta operación como merge, que se refiere al uso de algoritmos de combinación (merge algorithms), que son un conjunto de algoritmos que combinan listas ordenadas, en este caso se utilizarían con el operador and. La complejidad de realizar este proceso es, en el peor de los casos, de orden lineal, siendo crucial tener las listas ordenadas de forma global (usando el mismo criterio de ordenación para todas, el docID en este caso).

AND: Recupera documentos que contienen todos los términos especificados.

OR: Recupera documentos que contienen al menos uno de los términos especificados.

NOT: Recupera documentos que contienen el primer término pero no el segundo.

- Uso de índice posicional para responder a phrase queries

El uso de un índice posicional es fundamental para responder eficazmente a las consultas de frases (phrase queries). En una phrase query, el objetivo es encontrar documentos donde los términos aparezcan en el orden exacto especificado en la consulta. Un índice posicional no solo almacena los documentos donde aparecen los términos, sino también las posiciones exactas de esos términos dentro de cada documento, lo que permite verificar fácilmente si los términos aparecen en la secuencia requerida. De esta forma, si la primera palabra aparece en la posición x de un documento y la segunda en la posición x+1 del mismo documento, sabemos que van seguidas. Con este método se pueden localizar secuencias de cualquier número de palabras.

Un índice posicional para cada término en el corpus incluye una lista de documentos en los que aparece el término y, dentro de cada documento, las posiciones donde aparece.

- Indexado distribuido. Aproximación map reduce

Para el indexado a gran escala, por ejemplo en la web, las colecciones son tan grandes que es necesario usar clusters para poder procesarlas. Además, es mejor

usar varias máquinas “normales” que un computador de altas prestaciones, entre otras cosas porque un dispositivo individual es proclive a fallar y va a trabajar de forma más lenta. De esta forma surge la necesidad de encontrar formas de explotar conjuntos de máquinas para realizar la indexación. Es importante que el sistema sea tolerante a fallos, de forma que si falla una máquina otra pueda asumir su trabajo.

MapReduce:

Es un framework conceptual muy simple para computación distribuida. Se realizan dos tipos de tareas paralelas:

Parsers: leen las páginas y extraen las palabras, creando los pares (término, docID). Cada nodo parsea una página o conjunto de páginas. Se corresponde con la fase de map.

Inversores: crean las listas de cada término a partir de los pares proporcionados por los parsers. Cada nodo puede tener un dominio (a-f, g-l, ...). Correspondiente con la fase de reduce.

A la hora de realizar la división es importante llegar a un buen equilibrio entre segmentación y máquinas. Esto quiere decir que no se asignen bloques demasiado grandes y que el número total de bloques a gestionar no sea tampoco muy elevado.

Una vez realizado este proceso, hay que tener en cuenta las formas de dividir el índice:

Particionado por término: es una división vertical, en la que un nodo o máquina maneja un subconjunto de términos (vocabulario troceado).

Particionado por documento: división horizontal, un nodo maneja un subconjunto de documentos, pero todos los nodos tienen el vocabulario entero (postings lists troceadas).

- Ranked retrieval. Modelos de ponderación (tf/idf)

La recuperación clasificada. Este tipo de sistemas devuelven una lista ordenada de documentos para una consulta. Es un estilo muy popular en búsqueda web.

TF/IDF:

Para tener en cuenta la relevancia de los términos, se realiza la combinación de la frecuencia del término (tf, es el número de veces que un término t ocurre en un documento d .) y la frecuencia de documentos inversa (idf) para obtener como resultado un peso compuesto para cada término en cada documento.

Este método busca encontrar términos que sean frecuentes en un documento específico pero poco comunes en el conjunto de documentos en general. Esto ayuda a identificar términos distintivos del documento y pueden ser indicativos de su relevancia para una consulta dada.

Otra forma de verlo es que tf-idf asigna a t un peso en d que es:

1. Mayor cuando t se repite muchas veces en un número pequeño de documentos.
2. Menor cuando el término se repite pocas veces en el documento o se encuentra en muchos documentos.
3. Menor cuando t se repite en prácticamente todos los documentos.

A partir de esta medida podemos considerar la puntuación de un documento en función de una consulta q. Esta puntuación sería la suma, para todos los términos de la query, de la puntuación del término en el documento.

- Modelo vectorial de Recuperación de Información

Se conoce como modelo vectorial a la representación de un conjunto de documentos como vectores en un espacio vectorial común. Cada documento se representa como un vector de números reales, formado por pesos. La forma de cálculo de los pesos es una decisión externa al modelo, se puede calcular de la forma que se escoja, como por ejemplo con pesos tf-idf.

Las consultas se representan como vectores también, con la idea de medir la proximidad entre los vectores de documentos y consultas para responder a las búsquedas.

- Normalización por longitud de documento

Se puede normalizar la longitud de un vector dividiendo cada una de sus componentes por su longitud. Para convertir un vector en un vector unitario, se puede dividir por su norma L2. De esta forma, los documentos largos y cortos tendrán pesos comparables.

Fórmula: $\text{coseno}(\text{consulta}, \text{documento})$

$$||\vec{x}||_2 = \sqrt{\sum x_i^2}$$

- Modelo probabilística de RI (BIM, aspectos básicos, sin memorizar fórmulas). Saber estimar probabilidades p_i y r_i a partir de una tabla de ocurrencia de términos en docs relevantes e irrelevantes.

El uso de métodos probabilistas para la recuperación de información tiene como motivo principal la necesidad de clarificar la incertidumbre respecto a la relevancia de un documento. Este tipo de métodos proporcionan una base para poder razonar bajo la incertidumbre. Estos métodos son unos de los más antiguos pero también de los más usados actualmente.

BIM: Los documentos se representan como vectores de incidencia de términos, donde cada posición toma valor 1 si el término que representa esa posición está en el documento y 0 si no. De esta forma, documentos diferentes pueden tener un

mismo vector. Se modela considerando que los términos se presentan en los documentos de forma independiente (no se reconoce asociación entre términos). Aunque esta asunción no es muy correcta, suele generar resultados satisfactorios en la práctica.

p_i : Probabilidad de que el término i esté presente en un documento relevante.

r_i : Probabilidad de que el término i esté presente en un documento irrelevante.

- df_t es el número de documentos que contienen el término t .
- S es el número total de documentos relevantes.
- s es el número de documentos relevantes que contienen el término t .
- N es el número total de documentos en la colección.

	documento	relevante (R=1)	no relevante (R=0)	Total
término presente	$x_i = 1$	s	$df_t - s$	df_t
término ausente	$x_i = 0$	$S - s$	$(N - df_t) - (S - s)$	$N - df_t$
Total		S	$N - S$	N

$$p_i \approx \frac{s}{S}, \quad r_i \approx \frac{(df_t - s)}{(N - S)},$$

Término	Documentos Relevantes Conteniendo Término (n_{R_i})	Total Documentos Relevantes (R)	Documentos Irrelevantes Conteniendo Término (n_{I_i})	Total Documentos Irrelevantes (I)
Term1	10	20	5	80
Term2	15	20	10	80
Term3	5	20	40	80

Cálculo de Probabilidades p_i y r_i

Para cada término, calculamos p_i y r_i de la siguiente manera:

- p_i : Proporción de documentos relevantes que contienen el término.

$$p_i = \frac{n_{R_i}}{R}$$

- r_i : Proporción de documentos irrelevantes que contienen el término.

$$r_i = \frac{n_{I_i}}{I}$$

Ejemplo de Cálculo

- Term1:

$$p_1 = \frac{10}{20} = 0.5$$

$$r_1 = \frac{5}{80} = 0.0625$$

- BM25. Entender qué mejoras supone respecto a BIM.

BM25 (Best Matching 25) es una extensión y mejora del modelo probabilístico de recuperación de información (BIM). Fue desarrollado para abordar algunas de las limitaciones de BIM y proporcionar un enfoque más robusto para clasificar y recuperar documentos basados en su relevancia con respecto a una consulta. A continuación, se detallan las principales mejoras y características de BM25 en comparación con BIM.

Este método de scoring se basa en calcular puntuaciones de relevancia para cada documento en función de la frecuencia de los términos de consulta y la longitud del documento, y luego combinar estas puntuaciones para obtener una medida global de la relevancia del documento.

Mejoras Introducidas por BM25:

Frecuencia de Término (TF) Normalizada: BM25 considera la frecuencia de término, pero de una manera que se normaliza según la longitud del documento. Esto se logra mediante una fórmula

Inverso de la Frecuencia de Documentos (IDF) Mejorado: BM25 utiliza una versión refinada del IDF para asegurar que los términos que son raros en el corpus reciban un mayor peso

Parámetros Ajustables: BM25 introduce los parámetros k_1 y b para ajustar la contribución de la frecuencia de términos y la normalización de la longitud del documento, lo que permite una mayor flexibilidad y ajuste a diferentes tipos de corpora y consultas.

Ventajas de BM25:

Consideración de la Frecuencia de Términos: A diferencia de BIM, BM25 tiene en cuenta cuántas veces un término aparece en un documento, proporcionando una mejor estimación de la relevancia.

Normalización por Longitud de Documento: BM25 ajusta la frecuencia de términos según la longitud del documento, lo que ayuda a nivelar las diferencias entre documentos largos y cortos.

IDF Mejorado: La fórmula de IDF en BM25 es más sofisticada, mejorando la ponderación de términos raros y comunes.

Flexibilidad: Los parámetros ajustables k_1 y b permiten personalizar el modelo para diferentes necesidades y tipos de datos.

-Métricas de evaluación (y saberlas computar a partir de un conjunto o ranking de resultados): Precision, Recall, P@k, MAP, MRR, NDCG

Precisión:

La precisión mide la proporción de documentos recuperados que son relevantes.

$$\text{Precisión} = \frac{\text{Número de documentos relevantes recuperados}}{\text{Número total de documentos recuperados}}$$

Recall:

El recall mide la proporción de documentos relevantes que han sido recuperados del total de documentos relevantes disponibles.

$$\text{Recall} = \frac{\text{Número de documentos relevantes recuperados}}{\text{Número total de documentos relevantes}}$$

P@1:

P@k mide la precisión en los primeros k resultados devueltos por el sistema.

$$P@k = \frac{\text{Número de documentos relevantes en el top } k}{k}$$

MAP (Mean Average Precision):

MAP es el promedio de la precisión a través de diferentes niveles de recall. Se calcula primero la precisión en cada punto donde se recupera un documento relevante y luego se promedia.

$$\text{MAP} = \frac{1}{Q} \sum_{q=1}^Q \left(\frac{1}{n_q} \sum_{k=1}^{n_q} \text{Precisión}@k \right)$$

Mean Reciprocal Rank (MRR)

MRR mide la inversa de la posición del primer documento relevante en los resultados.

$$\text{MRR} = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{\text{rank}_i}$$

Si el primer documento relevante está en la posición 3:

$$\text{MRR} = \frac{1}{3} \approx 0.33$$

Normalized Discounted Cumulative Gain (NDCG):

NDCG mide la ganancia acumulada de los resultados ordenados, normalizada en función de la posición del documento y la relevancia. Se calcula en dos pasos:

1. Discounted Cumulative Gain (DCG):

$$DCG_p = \sum_{i=1}^p \frac{2^{\text{relevancia}_i} - 1}{\log_2(i + 1)}$$

2. NDCG:

$$NDCG_p = \frac{DCG_p}{IDCG_p}$$

- Evaluación mediante clicks (conceptos básicos).

No asumir clicks = relevancia, pero si clicks = preferencia

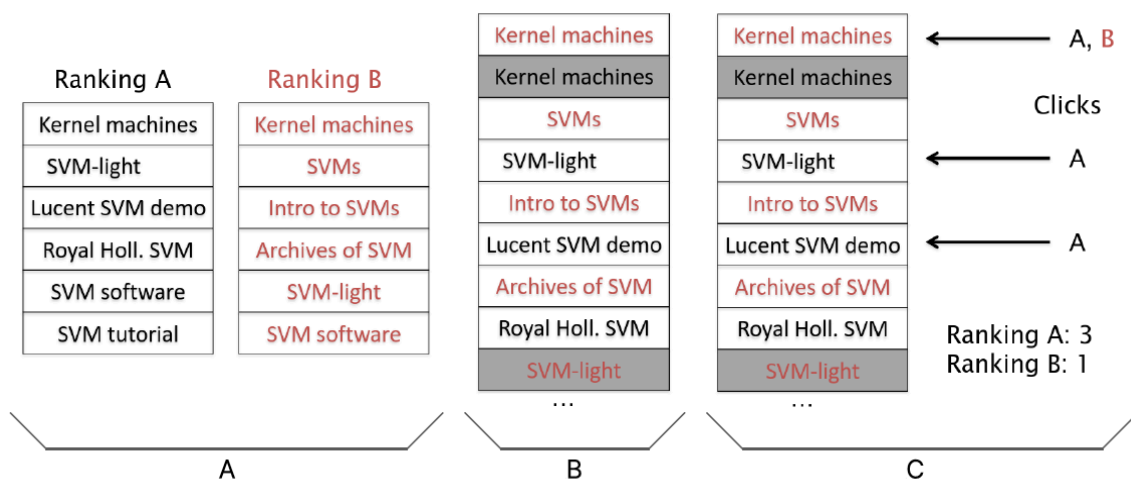
A la hora de analizar el orden de relevancia en función de los clicks, hay que tener en cuenta el orden en el que se muestran los resultados, ya que cuanto más arriba esté mayor probabilidad habrá de que se haga click en ellos. Esto se debe a los sesgos de los usuarios, que hacen que cuanto más arriba esté el documento más probabilidades haya de que se escoja. Por este motivo, no se considera cambiar la posición de un documento porque tenga muchos clicks, sino que se hace si tiene más clicks de los esperados para la posición de la lista en la que se encuentra.

También se puede analizar y usar el tiempo entre clicks, ya que, por ejemplo, si se selecciona un documento y a los 3 segundos se elige otro, probablemente el primero era muy malo. Puede haber mucho ruido (los tiempos largos pueden deberse a descansos del usuario o cualquier otra interrupción) pero es útil igualmente.

Otro análisis útil es comprobar el orden de navegación del usuario. Por ejemplo si hace click en el primer documento y más tarde en el tercero, se puede asumir que el tercero es mejor que el segundo. Esto solo se puede hacer con el inmediatamente anterior al seleccionado, ya que el usuario puede haberse olvidado de los anteriores (p.e. si se salta el segundo y el tercero se puede asumir que cuarto > tercero pero no que cuarto > segundo). Este método funciona bien para consultas populares (en las que hay muchos casos que poder tener en cuenta).

- Interleaved rankings y A/B testing

Una mejor solución consiste en entrelazar los resultados de ambos buscadores, de forma que se muestra el primer resultado de A, luego el primero de B, a continuación el segundo de A, y así sucesivamente. Una vez entrelazados los resultados, se eliminan los repetidos (los devueltos por ambos buscadores) y se muestra al usuario. Se suman puntos al buscador A por cada página de las que estaban en el ranking de A en la que se haya hecho click y lo mismo con B, sumando a ambos el punto si la página fue devuelta por ambos en la misma posición, o solo al que la pusiera en una posición más alta si ambos la devolvieron en posiciones diferentes. Se intercambia en cada consulta la lista que va primero (en el primer caso ABABAB..., en el segundo BABABA..., y así sucesivamente). Por último, se considera mejor el buscador que obtenga, de media, más puntuación.



A menudo los grandes motores como Google utilizan un porcentaje muy pequeño de usuarios para probar con ellos los nuevos cambios que van a realizar en los sistemas y comprobar su efectividad. Aunque el porcentaje de usuarios en el que lo prueban es muy pequeño, al ser motores tan usados supone un gran número de personas, por lo que resulta muy útil. Esta estrategia es conocida como A/B testing, y tiene como objetivo probar una innovación en el sistema. Es muy popular, especialmente en el ámbito de la recuperación de información. Para poder llevarlo a cabo, se necesita disponer de un motor de búsqueda ya en funcionamiento y con muchos usuarios, de forma que este será el sistema que utilicen la gran mayoría de usuarios (A), y un pequeño porcentaje se usará para realizar experimentos o evaluar una innovación (B). Si es posible realizar interleaving, se prueba de esa forma con los usuarios "experimentales" y, si no es posible (por ejemplo para cambios en la interfaz), se realiza el experimento completamente con la página nueva. Las pruebas de este tipo son conocidas como evaluación naturalista, porque los usuarios no son conscientes de que están participando en ellas y, por lo tanto, actúan de forma normal. Esto hace que sean muy útiles ya que reflejan el

comportamiento normal de los usuarios, no como otro tipo de pruebas vistas anteriormente.

- Clasificación de textos. Conceptos básicos. De qué tarea se trata la clasificación documental. VER APUNTES

La clasificación de texto es un caso particular de aprendizaje supervisado, ya que consiste en la predicción de una variable discreta (clases o temas). Es un área muy usada en diferentes ámbitos como, por ejemplo, el detector de correo basura. En la búsqueda web, la clasificación de texto tiene muchas aplicaciones, como la detección de páginas de spam, de contenido sexual explícito, o de comentarios positivos o negativos (p.e. búsquedas de reviews negativas de un producto). El problema consiste en, dada una representación de un documento (normalmente se usa algún tipo de espacio multidimensional o la bolsa de palabras) y un conjunto de clases con un tamaño fijo y conocido a priori, determinar la categoría del documento usando una función de clasificación. Es decir, se busca crear funciones de clasificación o clasificadores.

Existen diferentes métodos de clasificación:

- Clasificación manual: es útil para colecciones pequeñas, pero es muy difícil y caro de escalar.
- Reglas codificadas manualmente: la efectividad puede ser muy alta si la regla se ha ido refinando por un experto en este ámbito, pero construir y mantener este tipo de reglas es caro y encontrar expertos es difícil.
- Aprendizaje supervisado: en este caso, el criterio del clasificador se aprende automáticamente de datos de entrenamiento. Este tipo es el que se estudiara en este tema.

La clasificación documental se refiere a la tarea de asignar etiquetas a documentos enteros basándose en su contenido.

- Métodos de clasificación (reglas, aprendizaje supervisado con Naive Bayes, knn, Rocchio, etc)

-NaiveBayes: clasificación de texto y detección de spam. Usa métodos probabilistas basándose en pesos previos de las clases y parámetros condicionales de lo que dice cada palabra. Aprendizaje y test muy rápido y buenos resultados con características igual de importantes.

Robusto frente a características irrelevantes

→ Espacios vectoriales

training set se concibe como puntos etiquetados. Los documentos de una fase forman una región espacial continua y los diferentes no se superponen mucho. Delimitan clases en el espacio.

→ Rocchio: calcula centroide (vector medio) de cada clase y asigna a cada documento la clase del centroide más cercano. Centroide = media posiciones, no necesita el más cercano a todos.

Knn: obtiene Kvecinos más cercanos al documento a clasificar y le asigna la clase mayoritaria. No necesita entrenamiento.

Usar documentos de test como query => resultados => vecinos + cercanos

sensible a la calidad del training set.

- Métricas de rendimiento de clasificación (accuracy, etc).

Todas las de antes y:

F1 Score: La media armónica de la precisión y el recall.

- Link analysis. Anchor text. PageRank. HITS

El valor especial que añaden los enlaces es que aportan información que no es manipulable por los propietarios de la página. De todas formas, hay que tener en cuenta que siempre va a haber ruido, ya que se pueden crear, por ejemplo, granjas de enlaces. Un inconveniente a tener en cuenta con respecto a los enlaces es que la calidad de los mismos es muy diversa. Por este motivo, es importante diferenciar entre enlaces “buenos” y “malos”. Para hacer esto, se parte de las webs que se sabe que son “buenas” y “malas” para, mediante lógica simple iterativa, inferir de qué tipo son las desconocidas. El proceso de lógica simple iterativa asume los siguientes puntos:

Los nodos buenos no apuntan a nodos malos, por tanto:

- Si un nodo apunta a un nodo malo, es malo.
- Si un nodo bueno apunta a un nodo, ese nodo es bueno.

Anchor text:

Viendo la web como un grafo dirigido, surgen dos ideas básicas que son la razón de que los enlaces sean tan útiles para la búsqueda web. Estas dos ideas o hipótesis son las siguientes:

1. Un enlace entre páginas denota una concesión de autoridad o señal de calidad.
2. El texto descriptivo de un hiperenlace en una página (anchor text) describe a la página enlazada.

En resumen, un enlace aporta valor pero el texto asociado a ese enlace aporta también valor, siendo una señal de calidad o autoridad. Esto se basa en la idea de que el creador de una página B tiene poder sobre esa página pero no sobre todas las demás. Por este motivo no se usa solo lo que dice esta página B, sino que se añade también la información que aportan los demás sitios sobre B en el anchor text. Esta incorporación mejora mucho las búsquedas.

PageRank:

Este algoritmo está en el centro de buscadores como Google. Es una medida que toma valor entre 0 y 1 y se usa, en conjunto con otras de las métricas vistas anteriormente, para evaluar la calidad de las páginas web y poder así ordenarlas como resultado de una búsqueda. Se basa en medir la “visitabilidad” de una página, analizando la cantidad de veces que se llega a ella. Da importancia a los outlinks, haciendo que cuantas más veces sea citada una página (y a su vez cuanto más citadas sean las páginas que la enlazan) mayor pagerank tendrá. La base del funcionamiento de este algoritmo es la navegación por la web aleatoria (conocida como random walk), saltando de una página a otra mediante los enlaces disponibles y manteniendo un contador de visitas de cada página. De esta forma, desde una página hay la misma probabilidad de visitar cada una de las páginas a las que enlaza. Así, lo deseable para tener un alto pagerank es tener un número de inlinks elevados, preferiblemente de páginas con alto pagerank y pocos outlinks.

HITS:

Otro modelo de análisis de enlaces muy conocido es HITS, planteado por Kleinberg en 1999. A diferencia de pagerank, este método presenta dos factores de evaluación de calidad diferentes. Además, pagerank es independiente del usuario mientras que HITS responde a queries. Este modelo encaja mejor con temas de búsqueda amplios que con consultas para encontrar una página. La base de este método es que, en lugar de responder a una consulta con una lista ordenada de páginas, encuentra dos conjuntos de páginas interrelacionadas:

Hubs: son páginas que no resuelven una consulta por si mismas pero proporcionan enlaces a páginas relevantes. Un hub es bueno por los enlaces que proporciona.

Autoridades: son páginas que resuelven la consulta por si mismas. Una autoridad es buena por su propio contenido. Así, un buen hub para un tema enlaza a muchas autoridades para ese tema, mientras que una buena autoridad para un tema es apuntada por muchos hubs buenos.

- Conceptos básicos de crawling

El crawling es el proceso mediante el cual unos bots especializados van analizando y descubriendo páginas para indexarlas, con el objetivo de encontrar todas las páginas disponibles. Estos bots son un software muy simple que incluyen los buscadores, y son comúnmente conocidos como arañas o crawlers. También existe el focused crawling, que consiste en encontrar solo aquellas páginas que van sobre un tema a partir de la URL (saber cuales serán útiles y cuales no). En esta sección se hablará de crawling genérico. Algunos problemas a los que se enfrenta este

proceso son las páginas maliciosas, por un lado las de spam (lo ideal sería identificarlas ya en este proceso para evitar indexarlas) y, por otro lado, las denominadas trampas para arañas o spider traps, que generan enlaces dinámicos para atrapar a los bots en bucles infinitos. Los sitios no maliciosos también generan algunos problemas, por ejemplo las páginas dinámicas suponen un pequeño obstáculo, ya que hacen necesario un cierto nivel de inteligencia por parte de los bots. Otro reto pueden ser las páginas duplicadas o sitios espejo. Además de todo lo anterior, hay que considerar que la magnitud de esta operación hace imposible realizarla en una única máquina, por lo que es un proceso distribuido. La distribución se realiza también geográficamente, para poder optimizar en lo posible el ancho de banda y la cercanía entre peticiones.

Características de los crawlers

Condiciones necesarias:

- Ser robusto: no caer en trampas para arañas u otros comportamientos maliciosos de servidores web.
- Ser educado: respetar las consideraciones de educación explícitas e implícitas.

Además de estas dos condiciones necesarias, hay otras características esperables en un crawler que son las siguientes:

- Soportar operaciones distribuidas: estar diseñado para ejecutarse en múltiples máquinas distribuidas.
- Ser escalable: estar diseñado para aumentar el rango de crawl añadiendo nuevas máquinas.
- Ser eficiente: permitir el uso completo de los recursos de red y procesamiento disponibles.
- Indexar las páginas de “mayor calidad” primero.
- Realizar operaciones continuas: continuar reindexando copias nuevas de páginas previamente indexadas.
- Ser extensible: adaptarse a nuevos formatos de datos o protocolos.

PREGUNTAS VIQUEIRA

Teorema CAP

Fue formulado por Eric Brewer en 2000, en el ámbito de BBDD NoSQL se suele hablar de este teorema como la razón para relajar la consistencia.

El teorema CAP plantea que un sistema solo puede tener dos de las tres siguientes características:

- **Consistencia**

- **Disponibilidad:** si podemos comunicarnos con un nodo del cluster, entonces podemos leer y escribir datos.
- **Tolerancia a particionamiento:** el cluster puede sobrevivir a fallos de comunicación que lo separan en varias partes.

Un sistema centralizado tiene CA(Consistencia y Disponibilidad). Un servidor único no se puede particionar, por lo que no tiene tolerancia a particionado. Por otro lado, al ser único es consistente y si está levantado está disponible.

Un cluster puede ser CA, pero esto significa que, si se particiona en algún momento, debe pararse por completo, pero detener todos los nodos es muy costoso.

En realidad, los clusters deben tolerar particionamientos en la red. Y en este punto surge la aplicación real del teorema CAP: aunque en teoría es “solo puedes tener dos de las tres características”, en la práctica quiere decir que un sistema que puede sufrir particiones, como ocurre con los sistemas distribuidos, tiene que hallar un compromiso entre consistencia y disponibilidad (mejorar una empeora la otra).

La conclusión es que, aunque normalmente se busca consistencia, se puede relajar para conseguir disponibilidad y mejorar la eficiencia. Para poder llegar al mejor compromiso se necesita conocer el dominio de la aplicación, ya que en algunos casos es más importante la consistencia y, en otros, el rendimiento.

Se suele decir que los sistemas NoSQL siguen, en lugar de las propiedades ACID, las propiedades BASE. en lugar de compromiso entre consistencia y disponibilidad, es mejor (más claro) hablar de compromiso entre consistencia y latencia. Podemos pensar en la disponibilidad como el límite de latencia que podemos tolerar, cuando esta es demasiado alta se considera que el dato no está disponible. De esta forma, cuanto mayor consistencia (incluyendo más nodos en la interacción para mejorarla) menor latencia.

Modelado (clave-valor, documentales, etc) Basadas en grafos y column family
Los modelos clave-valor y documentales son fuertemente orientados a agregación dado que están construidas principalmente a través de agregados (un agregado es un conjunto de datos con el que interaccionamos como una unidad).

En las BBDD Clave-Valor el agregado es opaco para la base de datos, sin imponer restricciones sobre el contenido del agregado. Además, en estas BBDD solo se puede consultar por clave.

Las documentales son capaces de entender la estructura de los agregados, definiendo las estructuras posibles, lo cual añade una limitación pero aporta mayor flexibilidad en el acceso. Estas BBDD permiten obtener solo una parte del agregado, así como crear índices basados en su contenido.

El modelo column-family está formado por son estructuras tabulares con columnas dispersas y sin esquema, que tiene una estructura agregada en dos niveles. Se puede pensar en ellas como hashmaps anidados en dos niveles. Los datos se

estructuran usando cada fila como un agregado y las familias de columnas representan conjuntos de datos útiles de ese agregado. Cada familia de columnas define un tipo de registro, con filas para cada elemento. Estas familias de columnas proporcionan una cualidad bidimensional al modelo. Este modelo da cierta limitación de estructura al agregado pero permite a la BD aprovechar esa estructura para mejorar la accesibilidad.

El modelo basado en grafos es un caso especial de las BBDD NoSQL, busca el almacenamiento de registros simples con muchas relaciones entre los datos. Este modelo se basa en nodos y arcos, pudiendo almacenar datos tanto en unos como en otros. La ventaja que proporciona este modelo es la navegación rápida de las relaciones, aunque la inserción puede ser lenta

Replicación en BBDD

Replicación es tener los mismos datos copiados en diferentes nodos. Existen varias formas:

- Replicación maestro-esclavo: se replican los datos en varios nodos. Uno de los nodos se designa como maestro o primario, lo cual lo convierte en la autoridad como fuente de datos y el responsable de su actualización. Los demás nodos son esclavos o secundarios. Los nodos secundarios se sincronizan con el primario mediante el proceso de replicación, que es habitualmente asíncrono. Esta técnica es una buena solución cuando la aplicación es intensiva en lecturas, gracias a sus dos principales ventajas: Alta disponibilidad para lecturas y Sustitución del maestro ante un fallo. Inconvenientes: cuello de botella y puede haber fallos de consistencia.
- Replicación peer-to-peer: esta arquitectura es similar a la anterior pero eliminando la distinción entre maestro y esclavo. Esto se debe a que quiere solucionar alguno de los problemas de la anterior arquitectura(cuello de botella y baja disponibilidad). Uno de sus principales problemas es la consistencia(los write-write crean problemas que perduran). Para solucionarlo se puede: coordinar las réplicas durante la escritura (coste de red adicional para la coordinación y con actualizar la mayoría de forma coordinada sería suficiente) y asumir las inconsistencias e intentar arreglarlas combinando réplicas. Hay que tomar ciertas decisiones para llegar a un compromiso entre consistencia y disponibilidad.
- Combinación de particionamiento y replicación:
 - Con replicación maestro-esclavo: Un maestro único para cada partición y Dependiendo de la configuración se puede elegir maestro y esclavos a nivel de cluster o elegir maestro y esclavos para cada partición.

- Con replicación peer-to-peer: Muy común en soluciones NoSQL con modelos de tipo column-family y replicación con factor 3, cada partición tiene tres copias en tres nodos (si un nodo falla, las particiones de ese nodo se distribuyen entre los demás).